Oqab ESP32 Flight Controller

State Estimation, Multi-Rate Sensor Fusion, and Cascaded Control Architecture

Generated from `balance_controller` Firmware

November 2026

Abstract

This document describes the complete state estimation and flight control architecture implemented in the Oqab ESP32 quadrotor firmware (`balance_controller`). It details the dual-core FreeRTOS task split, sensor hardware and coordinate conventions, complementary and Kalman state estimators, optical flow derotation and latency compensation pipeline, autonomous flight mission state machine, PX4-style cascaded position and altitude outer loops, nine selectable inner attitude controllers, and the Quad-X motor mixer. TikZ flowcharts illustrate task execution, sensor fusion dataflows, state transitions, and cascaded feedback loops.

Contents

1 System Overview and Dual-Core Execution **2**

2 Hardware Sensors and Coordinate Conventions **2**
- 2.1 Sensor Suite and Interfaces . . . 2
- 2.2 Body Frame Sign Conventions and Lever Arms . . . 3

3 State Estimation Architecture **3**
- 3.1 Attitude Complementary Filter (250 Hz) . . . 3
- 3.2 3-State Vertical Kalman Filter (50 Hz) . . . 4
- 3.3 Horizontal 3-State Kalman Filters and Optical Flow Pipeline (50 Hz) . . . 5

4 Autonomous Flight Mission State Machine and Safety Guards **6**

5 Cascaded Control Architecture **6**
- 5.1 Outer Horizontal Position Cascade (50 Hz) . . . 6
- 5.2 Outer Vertical Altitude Cascade (50 Hz) . . . 7

6 Inner Attitude Controllers (250 Hz) and Flight Modes **7**
- 6.1 PID Controller (`CONTROLLER_PID`) . . . 7
- 6.2 Sliding Mode Controller (`CONTROLLER_SMC`) . . . 8
- 6.3 Linear Active Disturbance Rejection Control (`CONTROLLER_ADRC`) . . . 8
- 6.4 Integral Backstepping Family (`MDBCI, MDBCIS, CDI, CVI, NOI`) . . . 8
- 6.5 Variable Integral Adaptive Backstepping (`VIABC`) . . . 9

7 Quad-X Motor Mixing and Saturation **9**

1 System Overview and Dual-Core Execution

The Oqab flight controller runs on an ESP32 dual-core microcontroller using FreeRTOS to isolate deterministic real-time control from asynchronous UART parsing, telemetry serialization, and Flash NVS parameter storage.

- **Core 1 (`loop()`, Priority 15):** Executes the deterministic flight control pipeline at **250 Hz** ($\Delta t = 4000\,\mu$s), with nested time-triggered sub-loops at **50 Hz** (20 ms) for vertical/horizontal estimation and outer control loops, and **10 Hz** (100 ms) for optional GPS updates.
- **Core 0 (`CommsTask`, Priority 2, 8 KB stack):** Runs at **100 Hz** (10 ms period via `vTaskDelayUntil`) on the same core as Bluetooth SPP. It drains `Serial2` (MSPv2 optical flow and LiDAR, or NMEA GPS), processes CLI commands over USB Serial (115200 baud) and Bluetooth (`Oqab_Flight_Controller`), saves tuned parameters to ESP32 NVS Flash (`drone_params`), and emits telemetry at 20 Hz when disarmed or 50 Hz in simulation mode (muted when armed to prevent serial blocking).

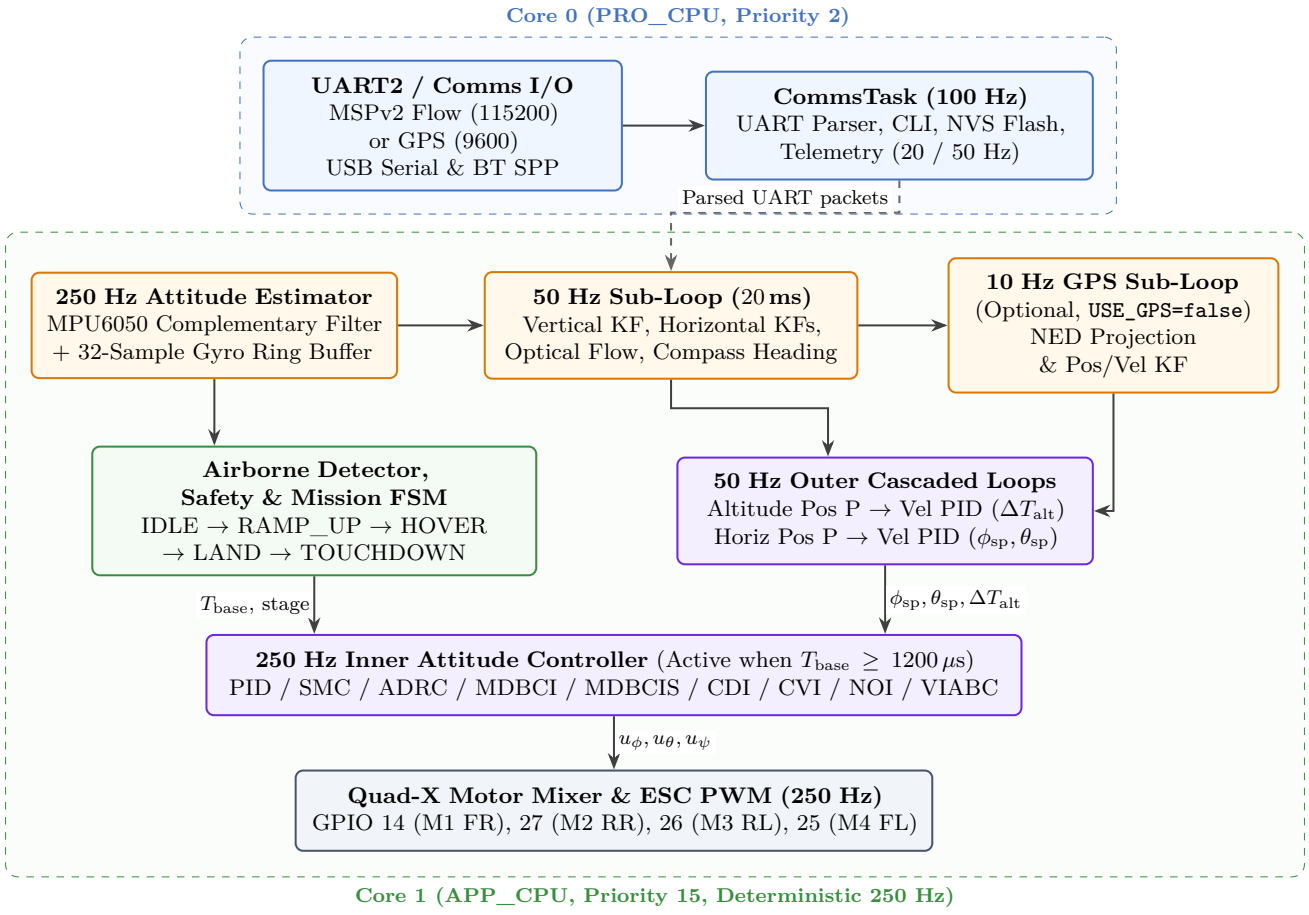


Figure 1: Dual-core FreeRTOS scheduling and multi-rate execution flow across Core 0 and Core 1.

2 Hardware Sensors and Coordinate Conventions

2.1 Sensor Suite and Interfaces

All sensors sit behind abstract C++ interfaces defined in `Sensors.h`: `IMUInterface`, `RangefinderInterface`, `BarometerInterface`, `CompassInterface`, `OpticalFlowInterface`, and `GPSInterface`. Table 1 summarizes the hardware drivers and buses.

Subsystem	Hardware / Driver	Bus / Rate	Role in Firmware
IMU	MPU6050 (`0x68`)	I2C 400 kHz, 250 Hz read	3-axis accel ($\pm 8g$), gyro ($\pm 500°/s$), 44 Hz DLPF
Optical Flow & ToF	Mateksys 3901-L0X	UART2 (RX 16, TX 17), 115200 baud	MSPv2 `0x1F02` flow, `0x1F01` VL53 ToF LiDAR (0.03–2.00 m)
Compass	GY-87 (QMC5883P / HMC5883L)	I2C bypass, 50 Hz read	Tilt-compensated True North yaw reference
GPS (Legacy)	NEO-M8N (NMEA)	UART2 shared, 9600 baud, 10 Hz	Disabled by default (`USE_GPS = false`)
Baro / Sharp IR	BMP180 / GP2Y0A	Dewired / Inactive	Bypassed when ToF LiDAR mode is enabled (`USE_LIDAR_ONLY_ALTITUDE`)

Table 1: Active sensor hardware and bus configuration defined in `Config.h` and `Sensors.cpp`.

2.2 Body Frame Sign Conventions and Lever Arms

The firmware uses a body-fixed frame where $+X$ points **Forward**, $+Y$ points **Right**, and $+Z$ points **Down** for physical lever-arm offsets relative to the center of mass (CM):

$$\mathbf{r}_{\text{flow}} = \begin{bmatrix} -0.04 \\ +0.01 \\ +0.03 \end{bmatrix} \text{m}, \qquad \mathbf{r}_{\text{imu}} = \begin{bmatrix} -0.02 \\ +0.02 \\ 0.00 \end{bmatrix} \text{m}. \tag{1}$$

From `StateEstimator.cpp`, gyroscope rates after calibration bias subtraction and axis sign mapping satisfy:

$$g_x = -(g_{x,\text{raw}} - b_{gx})\frac{180}{\pi} \quad \text{(right wing down} > 0\text{)}, \tag{2}$$

$$g_y = (g_{y,\text{raw}} - b_{gy})\frac{180}{\pi} \quad \text{(nose down} > 0\text{)}, \tag{3}$$

$$g_z = (g_{z,\text{raw}} - b_{gz})\frac{180}{\pi} \quad \text{(clockwise yaw} > 0\text{)}. \tag{4}$$

Correspondingly, positive roll $\phi > 0$ tilts the right wing down, positive pitch $\theta > 0$ pitches the nose down (accelerating forward), and positive yaw $\psi > 0$ rotates clockwise when viewed from above.

3 State Estimation Architecture

State estimation is partitioned into three estimators: a 250 Hz attitude complementary filter, a 50 Hz 3-state vertical Kalman filter, and two decoupled 50 Hz 3-state horizontal Kalman filters fused with latency-compensated optical flow (and optional 10 Hz GPS). Figure 2 summarizes the full estimation pipeline.

3.1 Attitude Complementary Filter (250 Hz)

At each 4 ms step, calibrated accelerometer angles are computed as:

$$\phi_{\text{acc}} = \text{atan2}\left(a_y, \sqrt{a_x^2 + a_z^2}\right)\frac{180}{\pi}, \qquad \theta_{\text{acc}} = \text{atan2}\left(a_x, \sqrt{a_y^2 + a_z^2}\right)\frac{180}{\pi}. \tag{5}$$

Roll and pitch fuse gyro integration with accelerometer tilt ($\alpha_{\text{att}} = 0.991$):

$$\phi_k = \alpha_{\text{att}}\left(\phi_{k-1} + g_x \Delta t\right) + (1 - \alpha_{\text{att}})\left(\phi_{\text{acc}} - \phi_{\text{cal}}\right), \tag{6}$$

$$\theta_k = \alpha_{\text{att}}\left(\theta_{k-1} + g_y \Delta t\right) + (1 - \alpha_{\text{att}})\left(\theta_{\text{acc}} - \theta_{\text{cal}}\right). \tag{7}$$

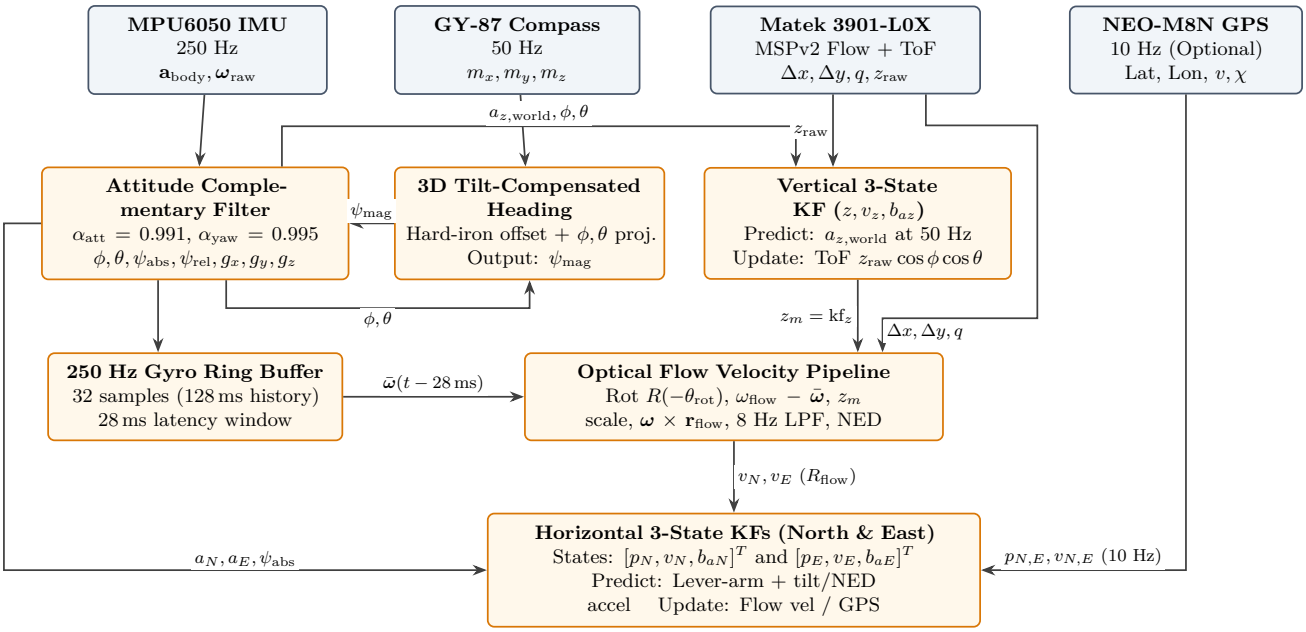


Figure 2: Multi-rate sensor fusion and state estimation dataflow across attitude, vertical, and horizontal estimators.

Compass readings (updated at 50 Hz) subtract hard-iron biases (m_{x0}, m_{y0}, m_{z0}) and project onto the horizontal plane using $(\phi + \phi_{\text{cal}}, \theta + \theta_{\text{cal}})$:

$$X_h = m_x \cos\theta + m_y \sin\phi \sin\theta + m_z \cos\phi \sin\theta, \qquad Y_h = m_y \cos\phi - m_z \sin\phi, \tag{8}$$

$$\psi_{\text{mag}} = \text{atan2}(Y_h, X_h)\frac{180}{\pi} + \delta_{\text{mag}}. \tag{9}$$

Absolute heading fuses gyro g_z with ψ_{mag} ($\alpha_{\text{yaw}} = 0.995$), wrapped to $[-180°, 180°]$:

$$\psi_{\text{abs},k} = \psi_{\text{abs},k-1} + g_z \Delta t + (1 - \alpha_{\text{yaw}})\ \text{wrap}_{180}(\psi_{\text{mag}} - \psi_{\text{abs},k-1}), \qquad \psi_{\text{rel}} = \text{wrap}_{180}(\psi_{\text{abs}} - \psi_{\text{offset}}). \tag{10}$$

Crucially, NED world-frame projections for horizontal estimation and position control use ψ_{abs}, whereas yaw damping uses body rate g_z.

3.2 3-State Vertical Kalman Filter (50 Hz)

The vertical estimator tracks $\mathbf{x}_z = [z,\ v_z,\ b_{az}]^T$ (altitude in m, climb rate in m/s, and vertical accelerometer bias in m/s^2). World vertical acceleration removes gravity along the tilted body axes:

$$a_{z,\text{world}} = a_x \sin\theta - a_y \sin\phi \cos\theta + a_z \cos\phi \cos\theta - 9.80665. \tag{11}$$

When airborne (or in simulation mode), the prediction step propagates with $a_{\text{corr}} = a_{z,\text{world}} - b_{az}$ and state transition matrix $\mathbf{F}_z$:

$$\mathbf{x}_{z,k|k-1} = \begin{bmatrix} 1 & \Delta t & -\frac{1}{2}\Delta t^2 \\ 0 & 1 & -\Delta t \\ 0 & 0 & 1 \end{bmatrix} \mathbf{x}_{z,k-1} + \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \\ 0 \end{bmatrix} a_{z,\text{world}}, \qquad \mathbf{Q}_z = \text{diag}(0.0002,\ 0.0035,\ 0.00003). \tag{12}$$

Prior to airborne liftoff confirmation, z tracks the ground baseline or valid ToF reading directly with $v_z = b_{az} = 0$ and primed covariance $\mathbf{P}_z = \text{diag}(0.01, 0.05, 0.01)$. The Mateksys VL53 ToF LiDAR measurement is tilt-compensated:

$$z_{\text{rf}} = z_{\text{raw}} \cos\phi \cos\theta, \tag{13}$$

and accepted when LiDAR quality > 30, $|\phi|, |\theta| \le 35°$, and $0.03 \le z_{\text{rf}} \le 2.00$ m. Scalar Kalman updates use measurement matrix $\mathbf{H} = [1\ 0\ 0]$ and variance $R_{\text{lidar}} = 0.0025\,\text{m}^2$, clamping $|b_{az}| \le 2.0\,\text{m/s}^2$.

3.3 Horizontal 3-State Kalman Filters and Optical Flow Pipeline (50 Hz)

Two identical 1D 3-state Kalman filters (`kfNorth`, `kfEast`) estimate $\mathbf{x}_h = [p,\ v,\ b_a]^T$ along Earth North and East using $\mathbf{Q}_h = \mathrm{diag}(0.005, 0.035, 0.0005)$.

Inertial Prediction Step

First, IMU centrifugal acceleration from yaw rate g_z (in rad/s) due to CM offset $(r_{\mathrm{imu},x}, r_{\mathrm{imu},y})$ is compensated, gravity projections are canceled, and calibrated static offsets are subtracted:

$$a_{x,\mathrm{cm}} = a_x + g_z^2 r_{\mathrm{imu},x}, \qquad a_{y,\mathrm{cm}} = a_y + g_z^2 r_{\mathrm{imu},y}, \tag{14}$$

$$a_{\mathrm{fwd}} = a_{x,\mathrm{cm}} \cos\theta - a_z \sin\theta - a_{\mathrm{fwd},0}, \qquad a_{\mathrm{right}} = a_{y,\mathrm{cm}} \cos\phi - a_z \sin\phi - a_{\mathrm{right},0}. \tag{15}$$

Rotating into Earth NED via ψ_{abs} gives $a_N = a_{\mathrm{fwd}} \cos\psi_{\mathrm{abs}} - a_{\mathrm{right}} \sin\psi_{\mathrm{abs}}$ and $a_E = a_{\mathrm{fwd}} \sin\psi_{\mathrm{abs}} + a_{\mathrm{right}} \cos\psi_{\mathrm{abs}}$. Inside `predict()`, bias-corrected acceleration $a_{\mathrm{corr}} = a_{\mathrm{meas}} - b_a$ applies a noise-floor deadband ($|a_{\mathrm{corr}}| < 0.015\,\mathrm{m/s^2} \Rightarrow 0$) and velocity clamp $|v| \le 6.0\,\mathrm{m/s}$. While resting on the ground, states are held at zero to prevent stationary drift.

Optical Flow Velocity Measurement Update

When a new MSPv2 `0x1F02` packet arrives with surface tracking quality $q \ge 35$, the optical flow estimator (`updateOpticalFlowEstimator()`) executes six stages:

1. **Latency-Matched Gyro Window:** Extracts delayed gyro averages $(\bar{g}_x, \bar{g}_y, \bar{g}_z)$ from the 250 Hz 32-sample ring buffer shifted back by round(FLOW_DELAY_MS/4 ms) = 7 samples (28 ms) across the flow integration window (≈ 5 samples for 20 ms).

2. **Mounting Rotation & Scaling:** Rotates raw counts (s_x, s_y) by $-\theta_{\mathrm{rot}}$ (default 90°) into body Forward/Right counts and scales by $s_{\mathrm{rad}} = 0.00380\,\mathrm{rad/count}$:

$$\omega_{\mathrm{flow},x} = \frac{s_x \cos\theta_{\mathrm{rot}} - s_y \sin\theta_{\mathrm{rot}}}{\Delta t}\, s_{\mathrm{rad}}, \qquad \omega_{\mathrm{flow},y} = \frac{s_x \sin\theta_{\mathrm{rot}} + s_y \cos\theta_{\mathrm{rot}}}{\Delta t}\, s_{\mathrm{rad}}. \tag{16}$$

3. **Gyro Derotation & Height Scaling:** Subtracts body angular rates (matching sign conventions: nose-down $\bar{g}_y$ and forward translation sweep ground features identically) and scales by clamped vertical KF height $z_m \in [0.05, 3.00]$ m:

$$v_{\mathrm{sens,fwd}} = (\omega_{\mathrm{flow},x} - \bar{g}_y) z_m, \qquad v_{\mathrm{sens,right}} = (\omega_{\mathrm{flow},y} - \bar{g}_x) z_m. \tag{17}$$

4. **Rigid-Body Lever-Arm Compensation:** Removes tangential velocity $\boldsymbol{\omega} \times \mathbf{r}_{\mathrm{flow}}$ caused by rotation about the CM (with nose-up rate $q_{\mathrm{up}} = -\bar{g}_y$):

$$v_{\mathrm{raw,fwd}} = v_{\mathrm{sens,fwd}} - (q_{\mathrm{up}} r_z - \bar{g}_z r_y), \qquad v_{\mathrm{raw,right}} = v_{\mathrm{sens,right}} - (\bar{g}_z r_x - \bar{g}_x r_z), \tag{18}$$

   clamped to ± 3.5 m/s.

5. **Low-Pass Filter & NED Rotation:** Filters body velocities with a 1st-order exponential LPF ($\alpha = 1 - e^{-2\pi f_c \Delta t}$, $f_c = 8.0$ Hz) and rotates by ψ_{abs} to (v_N, v_E).

6. **Quality-Adaptive Kalman Update:** Updates `kfNorth.updateVel` and `kfEast.updateVel` when airborne or in sim mode using quality-scaled variance:

$$R_{\mathrm{flow}} = \mathrm{clamp}\left(\frac{2R_{\mathrm{flow},0}}{q/255},\ 0.015,\ 0.40\right), \qquad R_{\mathrm{flow},0} = 0.02\,\mathrm{m^2/s^2}. \tag{19}$$

When GPS is enabled (`updatePositionEstimator()` at 10 Hz with ≥ 5 satellites and HDOP ≤ 4.0), flat-Earth lat/lon offsets update position with $R_{\mathrm{pos}} = \mathrm{clamp}(0.4\,\mathrm{HDOP}^2, 0.4, 15.0)$ and Doppler velocity updates velocity with $R_{\mathrm{vel}} = 0.08$.

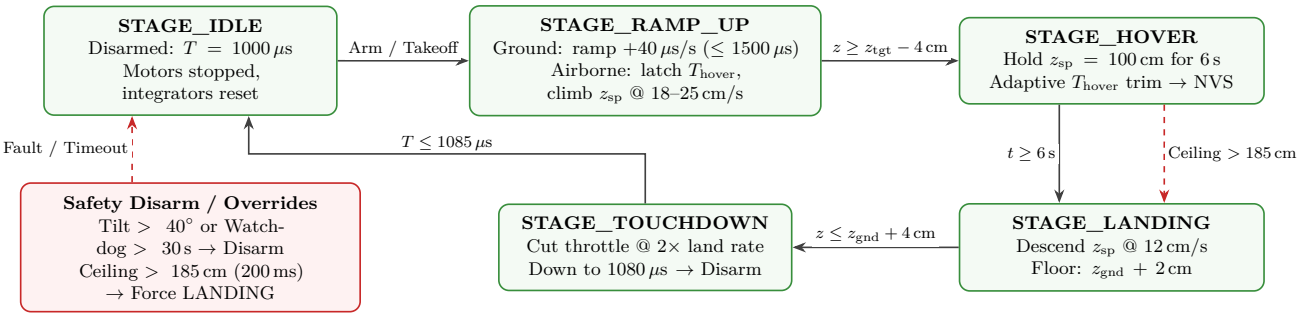


Figure 3: Autonomous flight mission state machine, airborne liftoff handoff, and safety overrides.

4 Autonomous Flight Mission State Machine and Safety Guards

Before the inner attitude loop computes motor commands, Core 1 evaluates airborne detection, safety overrides, and the 5-stage autonomous mission state machine (`balance_controller.ino`).
Key operational rules in the mission manager:

- **Debounced Airborne Confirmation:** While armed with $T_{base} \geq 1260\,\mu s$, liftoff is declared if fused height exceeds $z_{gnd} + 5\,cm$ or valid LiDAR exceeds $z_{gnd} + 4\,cm$ continuously for 150 ms (fallback: $T_{base} \geq 1360\,\mu s$ sustained for 600 ms). Upon confirmation, open-loop ramping stops and T_{hover} latches within $[1280, 1420]\,\mu s$.
- **In-Flight Hover Thrust Learning:** During `STAGE_HOVER`, whenever $|z - z_{tgt}| < 5\,cm$ and $|v_z| < 0.12\,m/s$, 5% per second of `alt_throttle_adj` is bled into $T_{hover} \in [1150, 1550]\,\mu s$ and saved to NVS parameter `MPC_THR_HOVER` after 2.0 s of steady hover.
- **Closed-Loop Activation & Anti-Windup:** Attitude controllers stay reset while $T_{base} < 1200\,\mu s$ (`PID_ACTIVE_THROTTLE`). Integral and ESO disturbance terms only accumulate when `run_iterm = sim_mode || is_airborne`.

5 Cascaded Control Architecture

Figure 4 shows the full cascaded control hierarchy connecting the 50 Hz outer position and altitude loops to the 250 Hz inner attitude loop and Quad-X mixer.

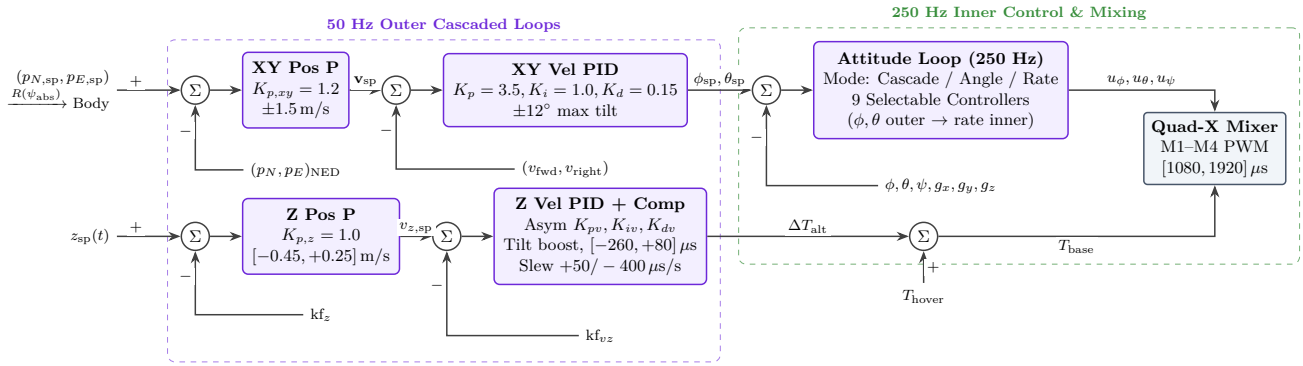


Figure 4: End-to-end cascaded flight control architecture from position/altitude outer loops to Quad-X motor mixing.

5.1 Outer Horizontal Position Cascade (50 Hz)

Implemented in `computePositionControl()` when position hold is active and either Optical Flow or a GPS home lock is available:

1. **Setpoint Latch & NED-to-Body Rotation:** Upon activation, the current NED position (p_N, p_E) is latched as $(p_{N,\text{sp}}, p_{E,\text{sp}})$. Position errors and filtered NED velocities are rotated into body Forward/Right coordinates via ψ_{abs}:

$$\begin{bmatrix} e_{\text{fwd}} \\ e_{\text{right}} \end{bmatrix} = \begin{bmatrix} \cos\psi_{\text{abs}} & \sin\psi_{\text{abs}} \\ -\sin\psi_{\text{abs}} & \cos\psi_{\text{abs}} \end{bmatrix} \begin{bmatrix} p_{N,\text{sp}} - p_N \\ p_{E,\text{sp}} - p_E \end{bmatrix}. \tag{20}$$

2. **Outer Position P Loop:** Desired body velocities are $v_{\text{sp}} = \text{clamp}(K_{p,xy} e, -v_{\max}, v_{\max})$ with $K_{p,xy} = 1.2\,\text{s}^{-1}$ and $v_{\max} = 1.5\,\text{m/s}$.

3. **Inner Velocity PID Loop:** Velocity errors $e_v = v_{\text{sp}} - v$ drive desired tilt angles (positive pitch nose-down accelerates forward; positive roll right-wing-down accelerates right):

$$\theta_{\text{sp}} = \text{clamp}(K_{p,v} e_{v,\text{fwd}} + I_{\text{fwd}} - K_{d,v} \dot{v}_{\text{fwd}},\ -\theta_{\max},\ \theta_{\max}), \tag{21}$$

with $K_{p,v} = 3.5°/(\text{m/s})$, $K_{i,v} = 1.0°/\text{m}$ (clamped at $\pm 7.0°$), $K_{d,v} = 0.15$ (derivative clamped $\pm 3.0°$), and $\theta_{\max} = 12.0°$. An identical law computes ϕ_{sp} on the Right axis.

5.2 Outer Vertical Altitude Cascade (50 Hz)

Implemented in `computeAltitudeControl()`:

1. **Outer Z Position P Loop:** $v_{z,\text{sp}} = \text{clamp}(K_{p,z}(z_{\text{sp}} - z), -v_{\text{desc,max}}, v_{\text{climb,max}})$ with $K_{p,z} = 1.0\,\text{s}^{-1}$, $v_{\text{climb,max}} = 0.25\,\text{m/s}$, and $v_{\text{desc,max}} = 0.45\,\text{m/s}$.

2. **Inner Z Velocity PID with Asymmetric Gains:** Using $e_{vz} = v_{z,\text{sp}} - v_z$:

$$a_{z,\text{sp}} = K_{p,\text{eff}}\, e_{vz} + I_{vz} - K_{d,vz} \frac{v_{z,k} - v_{z,k-1}}{\Delta t}, \tag{22}$$

$$K_{p,\text{eff}} = \begin{cases} K_{p,vz} = 1.8 & e_{vz} \geq 0 \text{ (climb)}, \\ 2K_{p,vz} = 3.6 & e_{vz} < 0 \text{ (brake/descend)}. \end{cases} \tag{23}$$

The integrator ($K_{i,vz} = 0.50$, clamped $[-2.5, 1.2]$) freezes positive accumulation and decays at 2.0/s whenever the drone is above target ($e_z < -0.04\,\text{m}$).

3. **Throttle Mapping, Tilt Compensation, and Asymmetric Slew Limit:** Acceleration maps to PWM via $\Delta T_{\text{raw}} = a_{z,\text{sp}} \cdot \frac{T_{\text{base}} - 1000}{9.80665}$, plus tilt-thrust boost $\left(\frac{1}{\cos\phi\cos\theta} - 1\right)(T_{\text{base}} - 1000)$ (clamped $+60\,\mu$s and attenuated above target). Authority is clamped asymmetrically to $[-260, +80]\,\mu$s (with extra emergency cut above $z_{\text{tgt}} + 25\,\text{cm}$ or $180\,\text{cm}$), then slew-limited to $+50\,\mu$s/s upward and $-400\,\mu$s/s downward ($-600\,\mu$s/s in ceiling emergency).

6 Inner Attitude Controllers (250 Hz) and Flight Modes

All nine attitude controllers inherit from `AttitudeController` and support three flight modes (`MODE_CASCADE` default, `MODE_ANGLE`, and `MODE_RATE`), switchable via `ctrl <name>`. Figure 5 shows the routing across flight modes and controller laws.

6.1 PID Controller (`CONTROLLER_PID`)

In `MODE_CASCADE`, outer proportional angle loops ($K_{p,\text{ang}} = 2.0$, clamped $\pm 400°/\text{s}$) generate rate targets ω_{sp} for inner rate PID loops ($K_p = 0.625, K_i = 2.1, K_d = 0.0088$):

$$u = K_p e_\omega + I_\omega - K_d \dot{g}_{\text{filt}}, \qquad \dot{g}_{\text{filt},k} = \dot{g}_{\text{filt},k-1} + 0.25\left(\frac{g_k - g_{k-1}}{\Delta t} - \dot{g}_{\text{filt},k-1}\right). \tag{24}$$

To suppress steady limit-cycling, I_ω decays by 0.96 per cycle when $|e_\theta| < 1.5°$ and saturates at ± 150. Yaw uses rate damping $u_\psi = K_{p,\psi}(\omega_{\psi,\text{sp}} - g_z)$ with $K_{p,\psi} = 2.0$.

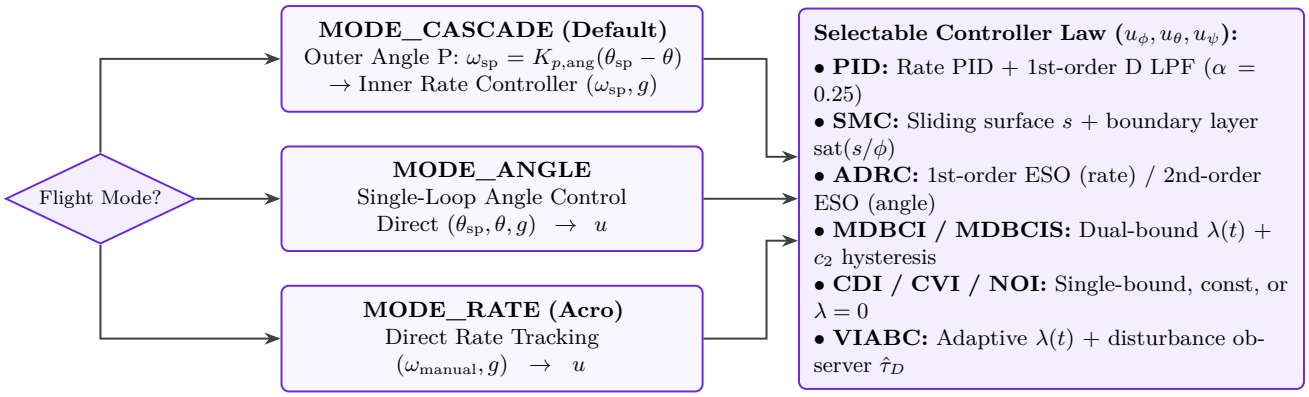


Figure 5: Flight mode routing and selectable 250 Hz inner attitude controller architectures.

6.2 Sliding Mode Controller (`CONTROLLER_SMC`)

In `MODE_CASCADE`, an outer P loop ($K_{p,\text{ang}} = 1.8$, clamped $\pm 220°/\text{s}$) feeds an integral sliding surface $s = e_\omega + \lambda_{\text{smc}} \int e_\omega dt$ ($\lambda_{\text{smc}} = 2.5$, integral clamped ± 35 with 0.96 decay inside 1.5°). To eliminate chattering, the switching term uses a continuous boundary layer ($\phi_{\text{smc}} = 20.0$):

$$u = K_{p,\text{smc}}\, s + K_{\text{sw}}\, \text{sat}\left(\frac{s}{\phi_{\text{smc}}}\right), \qquad \text{sat}(x) = \begin{cases} \text{sgn}(x) & |x| > 1, \\ x & |x| \le 1, \end{cases} \tag{25}$$

with defaults $K_{p,\text{smc}} = 0.50, K_{\text{sw}} = 28.0$ on roll/pitch and $K_{p,\text{smc}} = 1.5, K_{\text{sw}} = 20.0$ on yaw (no yaw integral).

6.3 Linear Active Disturbance Rejection Control (`CONTROLLER_ADRC`)

ADRC estimates total lumped disturbances (aerodynamic drag, CG imbalance, unmodeled torque) online using an Extended State Observer (ESO) and cancels them directly:

- **1st-Order Discrete LADRC (Cascade Inner Rate & Rate Mode):** State $\mathbf{z} = [\hat{\omega},\ \hat{d}]^T$ with observer bandwidth $\omega_o = 40\,\text{rad/s}$ ($l_1 = 2\omega_o, l_2 = \omega_o^2$), controller bandwidth $\omega_c = 15\,\text{rad/s}$ ($k_p = \omega_c$), and input gain $b_0 = 1.8$ (yaw rate: $b_0 = 1.0, \omega_c = 10, \omega_o = 30$):

$$u = \frac{k_p(\omega_{\text{sp}} - z_0) - z_1}{b_0}, \qquad \dot{z}_0 = z_1 + b_0 u + l_1(\omega - z_0), \qquad \dot{z}_1 = l_2(\omega - z_0). \tag{26}$$

- **2nd-Order Discrete LADRC (Angle Mode):** State $\mathbf{z} = [\hat{\theta},\ \hat{\omega},\ \hat{d}]^T$ with $\omega_o = 20, \omega_c = 6, \zeta = 1.2, b_0 = 1.8$ ($l_1 = 3\omega_o, l_2 = 3\omega_o^2, l_3 = \omega_o^3,\ k_p = \omega_c^2, k_d = 2\zeta\omega_c$):

$$u = \frac{k_p(\theta_{\text{sp}} - z_0) - k_d g - z_2}{b_0}, \qquad \dot{z}_0 = z_1 + l_1 e_{\text{obs}}, \quad \dot{z}_1 = z_2 + b_0 u + l_2 e_{\text{obs}}, \quad \dot{z}_2 = l_3 e_{\text{obs}}. \tag{27}$$

When `run_iterm = false`, the disturbance estimate z_1 (or z_2) is held at zero to avoid ground windup.

6.4 Integral Backstepping Family (`MDBCI, MDBCIS, CDI, CVI, NOI`)

Based on ISA Transactions 2020 (Eq. 44 and Section 4.2), let $z_1 = q_1 - q_d$ be the angle tracking error, $\chi_1 = \int z_1 dt \in [-20, 20]$ the integral error, q_2 the angular rate, and (q_v, q_a) the reference rate and acceleration. The Lyapunov backstepping control law is:

$$\tau = -(1 + \lambda_1 + c_1 c_2) z_1 - (c_1 + c_2)(q_2 - q_v) - c_2 \lambda_1 \chi_1 + q_a, \tag{28}$$

with defaults $c_1 = 1.8\,\text{rad/s}$ and $c_2 = 0.65\,\text{s}^{-1}$. The five variants differ solely in how the dynamic integral gain $\lambda_1(t)$ and damping c_2 evolve across outer boundary $\mu = 4.0°$ and inner boundary $\delta = 1.2°$ ($\lambda_r = 0.2, \lambda_m = 1.5, k_\lambda = 0.25, k_r = 0.1$):

Variant	Integral Gain Law $\lambda_1(t)$ and Damping Behavior
`NOI`	No integration: $\lambda_1 = 0$.
`CVI`	Conventional integral backstepping: $\lambda_1 = \lambda_m$ whenever airborne.
`CDI`	Conditional integration (Seshagiri & Khalil 2005): $\lambda_1 = \lambda_m$ if $\lvert z_1\rvert \le \mu$, else 0.
`MDBCI`	Dual-boundary conditional law: $\lambda_1 = 0$ for $\lvert z_1\rvert > \mu$; $\lambda_1 = \lambda_r$ for $\delta < \lvert z_1\rvert \le \mu$; for $\lvert z_1\rvert \le \delta$, $\dot{\lambda}_1 = k_\lambda z_1 + k_r \lvert q_v\rvert$ (transient) or $k_\lambda \lvert z_1\rvert + k_r \lvert q_v\rvert$ (steady hover), clamped to $[0, \lambda_m]$.
`MDBCIS`	Adds hysteresis switching ($h_z = 0.3°$): sets $c_2 = c_{2,\text{switch}} = 0.45$ when $\lvert z_1\rvert \le \delta$ and restores $c_2 = 0.65$ when $\lvert z_1\rvert > \delta + h_z$, mitigating motor jerk and sensor noise.

Table 2: Comparison of the five integral backstepping variants implemented in `MDBCI_Axis`.

6.5 Variable Integral Adaptive Backstepping (`VIABC`)

Based on IEEE Access 2024, `VIABC_Axis` updates $\lambda_1(t) \in [0, 2.5]$ outside a deadzone $\epsilon = 0.05°$ via $\dot{\lambda}_1 = \Gamma_1(\lvert z_1\rvert + \lvert q_v\rvert\, t_{ss})$ ($\Gamma_0 = 0.2, \Gamma_1 = 0.15$) and augments the backstepping torque with a disturbance observer $\hat{\tau}_D \in [-40, 40]$ driven by the virtual velocity error $z_2 = q_2 - (-c_1 z_1 - \lambda_1 \chi_1 + q_v)$:

$$\dot{\hat{\tau}}_D = -\Gamma_2 z_2 \;\; (\Gamma_2 = 0.05), \qquad \tau = -(1 + \lambda_1 + c_1 c_2) z_1 - (c_1 + c_2)(q_2 - q_v) - c_2 \lambda_1 \chi_1 + q_a + \hat{\tau}_D. \quad (29)$$

7 Quad-X Motor Mixing and Saturation

Attitude controller outputs are clamped to $\lvert u_\phi\rvert, \lvert u_\theta\rvert \le 350\,\mu$s (`MAX_PID_OUT`) and $\lvert u_\psi\rvert \le 200\,\mu$s (`MAX_YAW_OUT`), then combined with T_{base} in `Motors.cpp` at 250 Hz:

$$\begin{bmatrix} \text{PWM}_1 \text{ (FR, CCW, GPIO 14)} \\ \text{PWM}_2 \text{ (RR, CW, GPIO 27)} \\ \text{PWM}_3 \text{ (RL, CCW, GPIO 26)} \\ \text{PWM}_4 \text{ (FL, CW, GPIO 25)} \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 & -1 \\ 1 & -1 & +1 & +1 \\ 1 & +1 & +1 & -1 \\ 1 & +1 & -1 & +1 \end{bmatrix} \begin{bmatrix} T_{\text{base}} \\ u_\phi \\ u_\theta \\ u_\psi \end{bmatrix}. \quad (30)$$

Each ESC channel is constrained to $[1080, 1920]\,\mu$s while armed, or held at $1000\,\mu$s when disarmed or running in `sim_mode`.